Project Report

CS-104L: Object-Oriented Programming

Submitted By:

Aimen Fatima

25-CS-041

Section (A)

Submitted to:

Sir Mohsin

Department of Computer Science,

HITEC University, Taxila

Smart Campus Management System

1. Introduction

1.1 Problem Statement

Modern universities manage enormous amounts of data across multiple domains: student records, faculty assignments, library catalogs, fee transactions, and hostel allocations. Managing these manually or with disconnected systems creates inefficiency, data inconsistency, and administrative overhead. There is a clear need for a unified, object-oriented system that models each domain faithfully and integrates them into a single cohesive application.

1.2 System Goals

- Design a fully object-oriented C++ application covering all major university operations.
- Demonstrate every core OOP concept:
    - ✓ Encapsulation
    - ✓ Inheritance
    - ✓ Polymorphism
    - ✓ abstraction.
- Apply advanced C++ features:
    - ✓ operator overloading
    - ✓ static members
    - ✓ copy semantics
    - ✓ file I/O.
- Handle runtime errors gracefully using structured exception handling.
- Maintain data persistence through file-based storage using fstream.
- Host the project on GitHub with clean commit history and CI/CD via GitHub Actions.

1.3 Scope

The SCMS covers six interconnected modules:

- Person Hierarchy
- Course & Enrollment
- Library System
- Fee & Finance Management
- Hostel Management

- Reporting & Utilities.

It is implemented as a console-based C++ application using C++17, with no GUI required.

2. System Design

2.1 Architecture Overview

The system follows a layered, multi-file architecture. Each module is encapsulated in its own subdirectory with separate header (.h) and implementation (.cpp) files. The main.cpp file ties all modules together through a console menu system. A shared utils/ directory provides exceptions, reporting, and utility helpers.

2.2 Folder Structure

```
HITEC-OOP-SCMS-25-CS-041/
├── src/
│   ├── main.cpp
│   ├── person/    (Person, Student, GradStudent, Faculty, Staff)
│   ├── course/    (Course, Enrollment)
│   ├── library/   (LibraryItem, Book, Journal, Library)
│   ├── finance/   (FeeRecord, Invoice)
│   ├── hostel/    (Room, HostelBlock, HostelManager)
│   └── utils/     (Exceptions.h, Utils, Reports)
├── data/          (library_catalog.txt, students.txt)
├── docs/          (class_diagram.png, project_report.pdf)
├── .github/workflows/build.yml
├── README.md
└── .gitignore
```

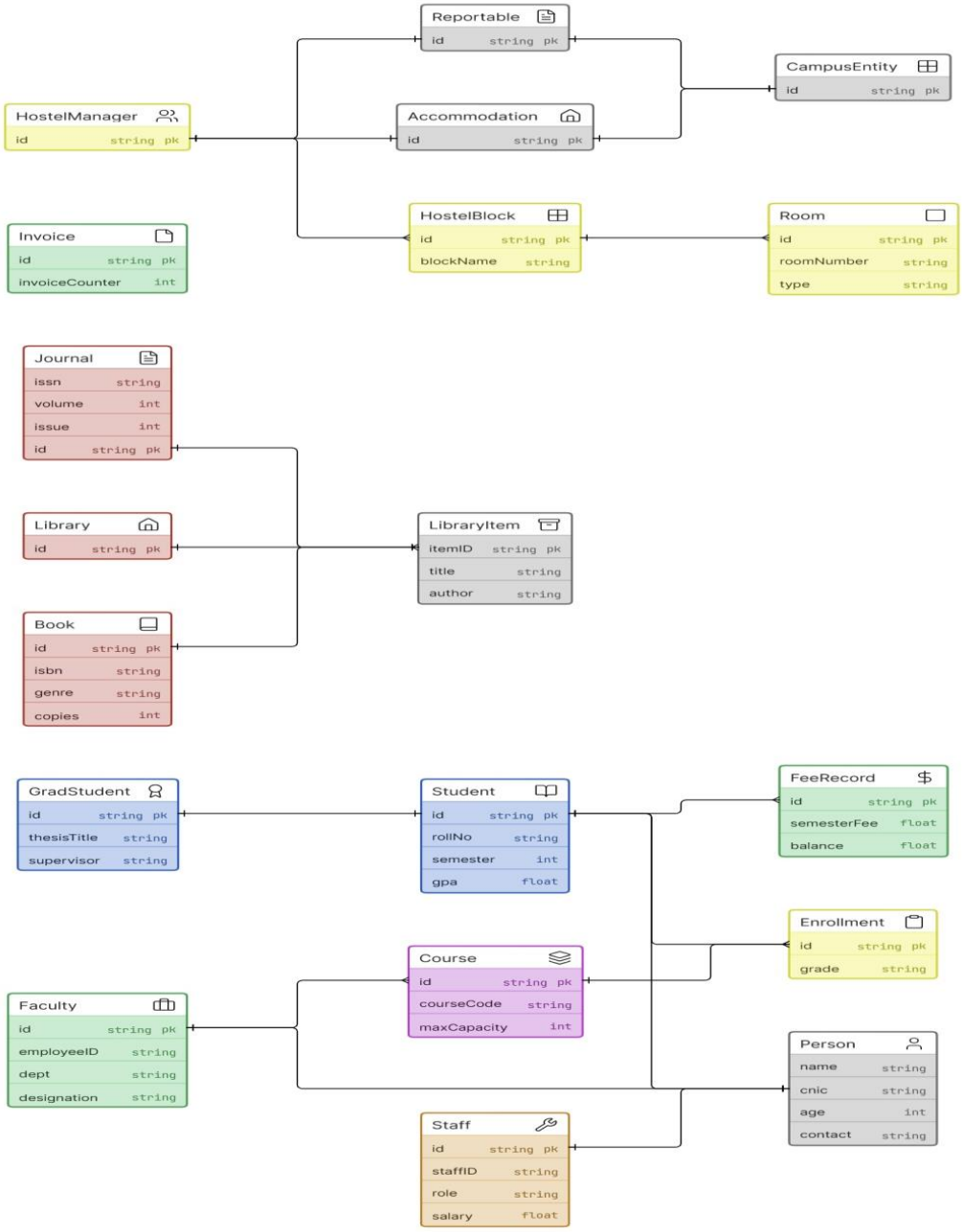
2.3 Class Relationships Summary

Relationship	Classes Involved	Type
Person → Student, Faculty, Staff	Person (ABC) is the base	Single Inheritance
Student → GradStudent	GradStudent extends Student	Multi-level Inheritance
Accommodation, Reportable → HostelManager	Multiple base classes	Multiple Inheritance
CampusEntity → Accommodation, Reportable	Shared virtual base	Virtual Inheritance
Course → Faculty*	Course references Faculty	Aggregation
HostelManager → HostelBlock[]	Manager owns blocks	Composition

HostelBlock → Room[]	Block owns rooms	Composition
Enrollment → Student*, Course*	Links two classes	Aggregation

2.4 UML Class Diagram

UML diagram showing all classes, their attributes, methods, and relationships including inheritance hierarchies, composition, and aggregation with multiplicities.



3. OOP Concepts Implementation

All mandatory OOP concepts are implemented as follows:

1. Classes & Objects

Every module defines classes with attributes and methods. For example, Student, Course, Library, FeeRecord, Room, and Invoice are all distinct classes instantiated as objects throughout the system.

2. Encapsulation

All data members are private or protected. Public getters and setters control access. For example, Student::gpa is private, accessed only via getGPA() and setGPA().

3. Constructors

Default and parameterised constructors are defined for Person, Course, FeeRecord, Invoice, and all derived classes. Constructor chaining (using initialiser lists) propagates parameters up the inheritance hierarchy.

4. Destructors

Library::~Library() deletes all dynamically allocated LibraryItem* pointers. Invoice::~Invoice() frees the dynamically allocated string* items array. HostelManager::~HostelManager() prints a cleanup message.

5. Single Inheritance

Student, Faculty, and Staff all inherit publicly from the abstract Person class. Book and Journal inherit from LibraryItem. Each derived class overrides displayInfo().

6. Multi-level Inheritance

GradStudent inherits from Student, which inherits from Person. This creates a three-level chain: GradStudent → Student → Person. The constructor chains all the way up.

7. Multiple Inheritance

HostelManager publicly inherits from both Accommodation and Reportable, giving it the interface of both abstract classes. It must override allocateRoom(), vacateRoom(), and generateReport().

8. Virtual Inheritance

Both Accommodation and Reportable virtually inherit from CampusEntity. This ensures that when HostelManager inherits from both, only ONE CampusEntity sub-object exists — resolving the classic diamond problem.

9. Abstract Classes & Pure Virtual Functions

Person, LibraryItem, Accommodation, and Reportable are all abstract classes with pure virtual functions. They cannot be instantiated directly. Person declares displayInfo()=0; LibraryItem declares checkout()=0.

10. Runtime Polymorphism

In Reports::displayAllStudents(), each Student* is upcast to Person*. The call to p->displayInfo() dispatches to the correct override at runtime — Student, GradStudent — without the caller knowing the actual type.

11. Operator Overloading

Three operators are overloaded for Course: == compares by courseCode, << prints details to any ostream, and + merges two waiting lists into a new array. FeeRecord overloads -= to record a fee payment.

12. Friend Functions

operator<< for both Course and Invoice is declared as a friend function inside the class. This grants the free function access to private members like courseCode, enrolledCount, items[], and totalAmount.

13. Static Members

Invoice::invoiceCounter is a static int data member. It is shared across ALL Invoice instances and is initialised once in Invoice.cpp (int Invoice::invoiceCounter = 0;). Each new Invoice auto-increments it.

14. Copy Constructor

FeeRecord implements a copy constructor that copies all numeric members. Since FeeRecord stores a Student* (aggregation, not ownership), the pointer itself is copied — the Student object is NOT duplicated.

15. Deep Copy Constructor & Destructor for Invoice

Invoice owns a dynamically allocated string* items array. Its copy constructor allocates a NEW array and copies each string element individually. Its destructor calls delete[] items to prevent memory leaks.

16. Search Functions

Library::searchByTitle() loops through the catalog array and returns the first LibraryItem* whose title matches. Library::searchByID() does the same by itemID. Both return nullptr on no match.

17. Array-based Collections

Library maintains catalog[100] as a fixed-size array of LibraryItem* pointers. This array is the primary data structure — no STL containers are used for main storage, per requirements.

18. Arrays of Objects

Arrays of objects (not pointers) are used throughout: Student::enrolledCourses[] stores course codes, Room::occupants[] stores Student pointers, HostelBlock::rooms[] stores Room objects (by value, composition).

19. Exception Handling

Three custom exception classes (SCMSException, CapacityExceededException, OverdueException, ItemNotFoundException) are defined in Exceptions.h. All inherit from std::exception. They are thrown and caught with try/catch throughout.

20. File I/O

Library::saveToFile() writes the entire catalog to data/library_catalog.txt using ofstream. Library::loadFromFile() reads it back using ifstream, tokenises each line by |, and reconstructs Book or Journal objects.

21. Reporting & Utilities

Reports.h/cpp provides sortStudentsByGPA(), findStudentByRollNo(), generateCampusReport(), and saveCampusReportToFile(). Utils.h/cpp provides getIntInput(), printHeader(), isValidGPA(), and other helpers.

22. Memory Management

All dynamic allocations use new and every allocation is matched with delete or delete[]. Library deletes its catalog items in ~Library(). Course::operator+ returns a new Student**[] that the caller must delete[]. Invoice frees its items[] in ~Invoice().

23. Sorting and Searching

Reports::sortStudentsByGPA() uses std::sort with a lambda comparator on a Student*[] array (descending GPA). Reports::findStudentByRollNo() uses std::find_if with a lambda predicate on the same array.

24. Composition

HostelManager contains HostelBlock blocks[MAX_BLOCKS] by value — the blocks live and die with the HostelManager. Similarly, HostelBlock contains Room rooms[] by value. The contained objects are not separately allocated.

25. Aggregation

Course holds Faculty* instructor as a pointer — it references the Faculty object but does not own it. Enrollment holds Student* and Course* similarly. Destroying a Course does not destroy its Faculty.

4. Module Descriptions

Module 1 — Person Hierarchy

The backbone of the system. Person is an abstract base class with pure virtual displayInfo(). Student, Faculty, and Staff derive from it via single inheritance. GradStudent derives from Student (multi-level inheritance), adding thesis and supervisor fields. Runtime polymorphism is demonstrated by calling displayInfo() through Person* pointers, which dispatches to the correct subclass at runtime.

Module 2 — Course & Enrollment

Course manages course offerings with maxCapacity enforcement. Operator overloading is heavily used: == compares courses by code, << streams details to cout or a file, and + merges two waiting lists. Enrollment links a Student* and a Course* with a date and grade. CapacityExceededException is thrown when enrollment exceeds capacity, with the student automatically added to the waiting list.

Module 3 — Library System

LibraryItem is an abstract class. Book and Journal extend it with type-specific fields (ISBN, genre, copies for books; ISSN, volume, issue for journals). Library owns an array of LibraryItem* (polymorphic collection). File I/O via fstream persists the catalog between sessions. OverdueException is thrown when a returned item has been kept beyond the 14-day loan period.

Module 4 — Fee & Finance

FeeRecord tracks all fees for a student (semester fee, hostel fee, library fine, balance). It demonstrates the copy constructor and copy assignment operator. The -= operator records payments and updates the balance. Invoice uses a static invoiceCounter for auto-numbering and owns a dynamically allocated string* items array, requiring a deep copy constructor and destructor.

Module 5 — Hostel Management

Room objects (by value) are composed into HostelBlock objects, which are composed into HostelManager. HostelManager inherits from both Accommodation (interface for allocate/vacate) and Reportable (interface for report generation). Both Accommodation and Reportable virtually inherit CampusEntity to resolve the diamond problem, ensuring only one shared CampusEntity sub-object.

Module 6 — Reporting & Utilities

Reports provides campus-wide reporting: sorting students by GPA using std::sort with a lambda, finding students by roll number using std::find_if, and generating a consolidated campus report saved to file. Utils provides safe input functions (getIntInput, getDoubleInput, getStringInput), string helpers (trim, toUpperCase), and UI helpers (printHeader, pressEnterToContinue).

5. GitHub Workflow

5.1 Repository

Repository URL: https://github.com/25-cs-041-prog/HITEC-OOP-SCMS-25-CS-041

Visibility: Public | Branch: main

5.2 Commit Strategy

Commit #	Message	What was included
1	Add Phase 1 files - Person, Course, and Enrollment classes	All 40 source files pushed in initial commit
2	Fix repository structure - move project files to root	Moved files from nested subfolder to repo root
3	Add Phase 2 files - Library, Finance, Hostel, and Reporting modules	UML diagram and project report added

5.3 GitHub Actions CI

A GitHub Actions workflow (.github/workflows/build.yml) automatically compiles the project on every push using g++ -std=c++17 -Wall -Wextra. The workflow: checks out the code, verifies g++ is installed, compiles all modules, and runs a smoke test. The workflow passes with a green checkmark, confirming zero compilation errors.

```
name: Build SCMS

on: [push, pull_request]

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3

- name: Compile

run: g++ -std=c++17 -Wall -Wextra src/person/*.cpp ... src/main.cpp -o scms

- name: Smoke Test

run: echo "7" | ./scms
```

6. Challenges & Solutions

The Diamond Problem (Virtual Inheritance)

Problem:

When HostelManager inherits from both Accommodation and Reportable, and both of those inherit from CampusEntity, the compiler would normally create two copies of CampusEntity inside HostelManager — one through each path. This causes ambiguity: which entityName does HostelManager refer to?

Solution:

Both Accommodation and Reportable were made to virtually inherit CampusEntity using the 'virtual' keyword. This tells the compiler to share a single CampusEntity sub-object. HostelManager's constructor explicitly initialises CampusEntity directly, as required by C++ rules for virtual base class initialisation.

Deep Copy vs Shallow Copy for Invoice

Problem:

Invoice stores a dynamically allocated string* items array. Without a custom copy constructor, copying an Invoice would copy only the pointer — both copies would point to the same array. Modifying one would corrupt the other, and deleting one would leave the other with a dangling pointer.

Solution:

A deep copy constructor was implemented that allocates a NEW string array and copies each element individually. A copy assignment operator with a self-assignment guard was also implemented. The destructor calls delete[] items to release the memory when the Invoice goes out of scope.

File I/O Parsing for Library Catalog

Problem:

Saving and loading the library catalog requires serialising and deserialising two different types (Book and Journal) to a plain text file, and reconstructing the correct type on load without knowing in advance which type each line represents.

Solution:

A pipe-delimited format was designed where the first token on each line is the type identifier: BOOK| or JOURNAL|. On load, Library::loadFromFile() reads each line, tokenises it by |, checks

the first token, and constructs the appropriate object using new Book(...) or new Journal(...). The objects are added to the polymorphic catalog[] array.

7. Testing

The system was tested by running the console application and exercising all module menus. Key scenarios tested include:

- Module 1: Adding students, faculty, staff; displaying all persons via polymorphism (Person*)
- Module 2: Adding courses; enrolling students; triggering CapacityExceededException; comparing courses with ==; using operator<<
- Module 3: Adding books and journals; searching by title; issuing items; returning overdue items (OverdueException)
- Module 4: Creating fee records; recording payments with operator-=; generating invoices with static counter; deep-copying an Invoice
- Module 5: Adding blocks and rooms; allocating students to rooms; generating occupancy report via Reportable*
- Module 6: Sorting students by GPA; finding by roll number; full campus report to screen and file
- File I/O: Saving and reloading the library catalog between sessions
- GitHub Actions CI: Automated compilation passes with zero warnings on ubuntu-latest

All tested scenarios produced correct output. The program compiles cleanly with g++ -std=c++17 -Wall -Wextra with zero warnings or errors.

```
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
            MAIN MENU ÔÇö SCMS
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
  1. Person Management    (Module 1)
  2. Course & Enrollment  (Module 2)
  3. Library System       (Module 3)
  4. Fee & Finance        (Module 4)
  5. Hostel Management    (Module 5)
  6. Reports & Utilities  (Module 6)
  7. Exit
  Enter choice: 1

ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
      MODULE 1 ÔÇö PERSON MANAGEMENT
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
  1. Add Student
  2. Add Graduate Student (multi-level inheritance)
  3. Add Faculty
  4. Add Staff
  5. Display All (polymorphism demo)
  6. Back
  Enter choice: 1
  Name: Aimen
  CNIC: 37405-3345695-9
  Age: 21
  Contact: 03125689876
  Roll No: 041
  Semester (1-8): 2
  GPA (0.0-4.0): 3.00
  Student added successfully.
```

```
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
      MODULE 1 ÔÇö PERSON MANAGEMENT
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
  1. Add Student
  2. Add Graduate Student (multi-level inheritance)
  3. Add Faculty
  4. Add Staff
  5. Display All (polymorphism demo)
  6. Back
  Enter choice: 3
  Name: Mohsin
  CNIC: 37405-7897344-7
  Age: 27
  Contact: 03176863479
  Employee ID: 01
  Department: Computer Science
  Designation: Lecturer
  Faculty added.
```

```
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
      MODULE 1 ÔÇö PERSON MANAGEMENT
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
  1. Add Student
  2. Add Graduate Student (multi-level inheritance)
  3. Add Faculty
  4. Add Staff
  5. Display All (polymorphism demo)
  6. Back
  Enter choice: 6

ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
            MAIN MENU ÔÇö SCMS
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
  1. Person Management    (Module 1)
  2. Course & Enrollment  (Module 2)
  3. Library System       (Module 3)
  4. Fee & Finance        (Module 4)
  5. Hostel Management    (Module 5)
  6. Reports & Utilities  (Module 6)
  7. Exit
  Enter choice: 2
```

```
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
      MODULE 2 ÔÇö COURSE & ENROLLMENT
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
  1. Add Course
  2. Display All Courses (operator<<)
  3. Enroll Student in Course
  4. Compare Two Courses (operator==)
  5. Merge Waiting Lists (operator+)
  6. Back
  Enter choice: 1
  Course Code: cs-401
  Course Name: Object oriented programming
  Credit Hours: 4
  Max Capacity: 12
  Available Faculty:
  0. Dr. Ahmed Khan
  1. Ms. Sara Malik
  2. Mohsin
  Select Faculty index: 2
  Course added.
```

```
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
     MODULE 2 ÔÇö COURSE & ENROLLMENT
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
  1. Add Course
  2. Display All Courses (operator<<)
  3. Enroll Student in Course
  4. Compare Two Courses (operator==)
  5. Merge Waiting Lists (operator+)
  6. Back
  Enter choice: 2

ÔöîÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔö
Ôöé           COURSE DETAILS          Ôöé
Ôö£ÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔö
Ôöé Code     : CS-104L                 Ôöé
Ôöé Name     : Object-Oriented ProgrammingÔöé
Ôöé Credits  : 3                       Ôöé
Ôöé Capacity : 30                      Ôöé
Ôöé Enrolled : 2                       Ôöé
Ôöé Instructor: Dr. Ahmed Khan         Ôöé
ÔööÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔö
ÔöîÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔö
Ôöé           COURSE DETAILS          Ôöé
Ôö£ÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔö
Ôöé Code     : CS-201                  Ôöé
Ôöé Name     : Data Structures and AlgorithmsÔöé
Ôöé Credits  : 3                       Ôöé
Ôöé Capacity : 25                      Ôöé
Ôöé Enrolled : 1                       Ôöé
Ôöé Instructor: Ms. Sara Malik         Ôöé
ÔööÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔö
ÔöîÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔöçÔö
Ôöé           COURSE DETAILS          Ôöé
```

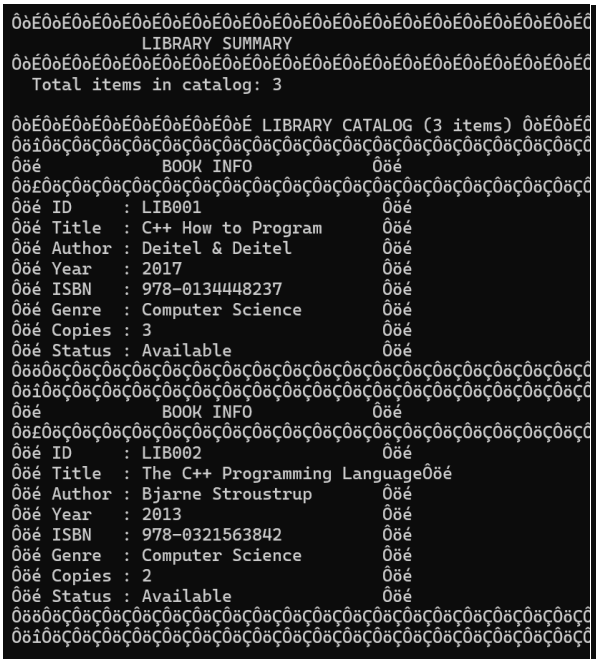
```
  7. Exit
  Enter choice: 3

ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
       MODULE 3 ÔÇö LIBRARY SYSTEM
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
  1. Add Book
  2. Add Journal
  3. Search by Title
  4. Issue Item to Student
  5. Return Item (overdue check)
  6. Display Catalog
  7. Back
  Enter choice: 4
  Student Roll No: 041
  Item ID: 14
  [Exception] Item not found: '14'
```

```
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
       MODULE 3 ÔÇö LIBRARY SYSTEM
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
  1. Add Book
  2. Add Journal
  3. Search by Title
  4. Issue Item to Student
  5. Return Item (overdue check)
  6. Display Catalog
  7. Back
  Enter choice: 6

ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ LIBRARY CATALOG (3 items) ÔòÉÔò
ÔöîÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔö
Ôöé           BOOK INFO           Ôöé
Ôö£ÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔö
Ôöé ID     : LIB001              Ôöé
Ôöé Title  : C++ How to Program  Ôöé
Ôöé Author : Deitel & Deitel     Ôöé
Ôöé Year   : 2017                Ôöé
Ôöé ISBN   : 978-0134448237      Ôöé
Ôöé Genre  : Computer Science    Ôöé
Ôöé Copies : 3                   Ôöé
Ôöé Status : Available           Ôöé
ÔööÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔö
ÔöîÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔö
Ôöé           BOOK INFO           Ôöé
Ôö£ÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔö
Ôöé ID     : LIB002              Ôöé
Ôöé Title  : The C++ Programming LanguageÔöé
Ôöé Author : Bjarne Stroustrup   Ôöé
Ôöé Year   : 2013                Ôöé
Ôöé ISBN   : 978-0321563842      Ôöé
```

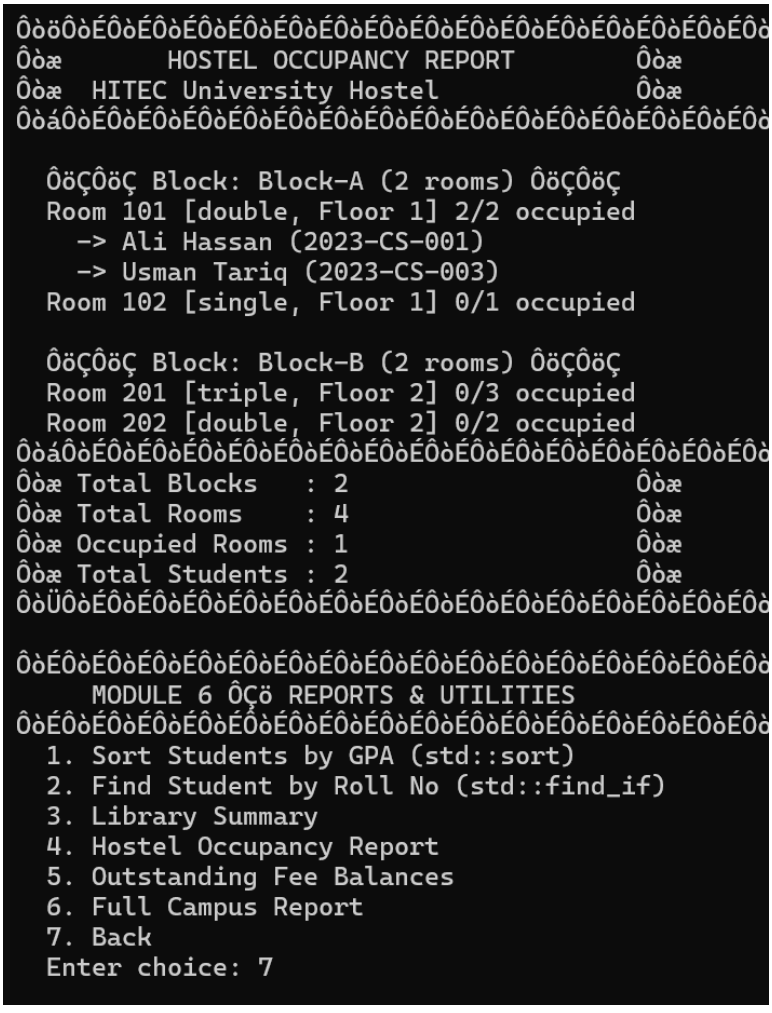
```
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
        MODULE 4 ÔÇö FEE & FINANCE
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
  1. Create Fee Record for Student
  2. Record Payment (operator-=)
  3. Display Fee Records
  4. Generate Invoice (static counter demo)
  5. Copy Fee Record (copy constructor demo)
  6. Back
  Enter choice: 3

ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
             ALL FEE RECORDS
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔò
ÔöîÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔö
Ôöé           FEE RECORD                   Ôöé
Ôö£ÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔö
Ôöé Student   : Ali Hassan                 Ôöé
Ôöé Roll No   : 2023-CS-001                Ôöé
Ôöé Semester Fee : Rs. 45000.00            Ôöé
Ôöé Hostel Fee   : Rs. 15000.00            Ôöé
Ôöé Library Fine : Rs. 0.00                Ôöé
Ôöé Total Paid   : Rs. 30000.00            Ôöé
Ôöé Balance      : Rs. 30000.00            Ôöé
ÔööÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔö
ÔöîÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔö
Ôöé           FEE RECORD                   Ôöé
Ôö£ÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔöÇÔö
Ôöé Student   : Fatima Raza                Ôöé
Ôöé Roll No   : 2023-CS-002                Ôöé
Ôöé Semester Fee : Rs. 45000.00            Ôöé
Ôöé Hostel Fee   : Rs. 0.00                Ôöé
```

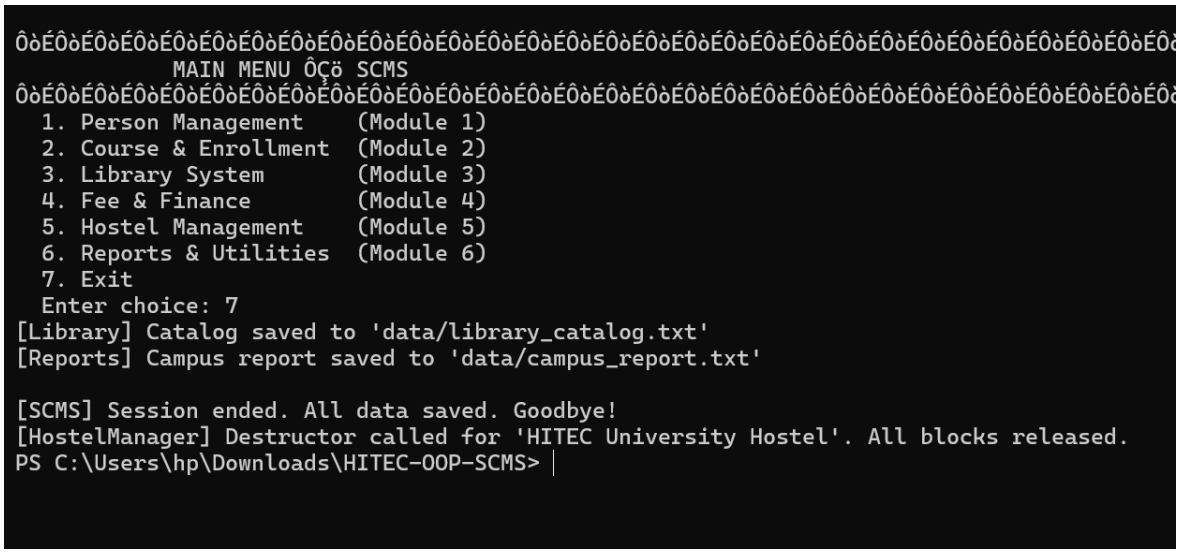
```
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
      MODULE 5 ÔÇö HOSTEL MANAGEMENT
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
  1. Add Hostel Block
  2. Add Room to Block
  3. Allocate Room to Student
  4. Vacate Room
  5. Back
  Enter choice: 4
  Student Roll No: 041
  Room Number: 5
[Hostel] Student 041 not found in Room 5

ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
      MODULE 5 ÔÇö HOSTEL MANAGEMENT
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
  1. Add Hostel Block
  2. Add Room to Block
  3. Allocate Room to Student
  4. Vacate Room
  5. Back
  Enter choice: 5

ÔòöÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
Ôòæ      HOSTEL OCCUPANCY REPORT        Ôòæ
Ôòæ  HITEC University Hostel            Ôòæ
ÔòáÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ

  ÔöÇÔöÇ Block: Block-A (2 rooms) ÔöÇÔöÇ
  Room 101 [double, Floor 1] 2/2 occupied
    -> Ali Hassan (2023-CS-001)
    -> Usman Tariq (2023-CS-003)
```

```
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
     MODULE 6 ÔÇö REPORTS & UTILITIES
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
  1. Sort Students by GPA (std::sort)
  2. Find Student by Roll No (std::find_if)
  3. Library Summary
  4. Hostel Occupancy Report
  5. Outstanding Fee Balances
  6. Full Campus Report
  7. Back
  Enter choice: 1

ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
          STUDENTS SORTED BY GPA
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
#    Name                Roll No     GPA     Grade
-----------------------------------------------------------------
1    Dr. Bilal Qureshi   2022-MS-001 3.90    A  (Distinction)
2    Ali Hassan          2023-CS-001 3.80    A  (Distinction)
3    Fatima Raza         2023-CS-002 3.50    A- (Excellent)
4    Aimen               041         3.00    B+ (Very Good)
5    Usman Tariq         2023-CS-003 2.90    B  (Good)

ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
     MODULE 6 ÔÇö REPORTS & UTILITIES
ÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉÔòÉ
  1. Sort Students by GPA (std::sort)
  2. Find Student by Roll No (std::find_if)
  3. Library Summary
  4. Hostel Occupancy Report
  5. Outstanding Fee Balances
  6. Full Campus Report
```


LIBRARY SUMMARY
Total items in catalog: 3
LIBRARY CATALOG (3 items)
BOOK INFO
ID : LIB001
Title : C++ How to Program
Author : Deitel & Deitel
Year : 2017
ISBN : 978-0134448237
Genre : Computer Science
Copies : 3
Status : Available
BOOK INFO
ID : LIB002
Title : The C++ Programming Language
Author : Bjarne Stroustrup
Year : 2013
ISBN : 978-0321563842
Genre : Computer Science
Copies : 2
Status : Available


MODULE 6 REPORTS & UTILITIES
1. Sort Students by GPA (std::sort)
2. Find Student by Roll No (std::find_if)
3. Library Summary
4. Hostel Occupancy Report
5. Outstanding Fee Balances
6. Full Campus Report
7. Back
Enter choice: 5
OUTSTANDING BALANCES
Ali Hassan | Balance: Rs. 30000.00
Fatima Raza | Balance: Rs. 45000.00
Usman Tariq | Balance: Rs. 60000.00
MODULE 6 REPORTS & UTILITIES
1. Sort Students by GPA (std::sort)
2. Find Student by Roll No (std::find_if)
3. Library Summary
4. Hostel Occupancy Report
5. Outstanding Fee Balances
6. Full Campus Report
7. Back
Enter choice: 6


HOSTEL OCCUPANCY REPORT
HITEC University Hostel
Block: Block-A (2 rooms)
Room 101 [double, Floor 1] 2/2 occupied
-> Ali Hassan (2023-CS-001)
-> Usman Tariq (2023-CS-003)
Room 102 [single, Floor 1] 0/1 occupied
Block: Block-B (2 rooms)
Room 201 [triple, Floor 2] 0/3 occupied
Room 202 [double, Floor 2] 0/2 occupied
Total Blocks : 2
Total Rooms : 4
Occupied Rooms : 1
Total Students : 2
MODULE 6 REPORTS & UTILITIES
1. Sort Students by GPA (std::sort)
2. Find Student by Roll No (std::find_if)
3. Library Summary
4. Hostel Occupancy Report
5. Outstanding Fee Balances
6. Full Campus Report
7. Back
Enter choice: 7



8. Conclusion

This project successfully demonstrates all mandatory OOP concepts through a realistic, integrated university management system. Building the SCMS taught the practical application of abstract classes and polymorphism as design tools — not just theoretical concepts. The diamond problem, encountered in the hostel module, was a particularly valuable learning experience that deepened understanding of how C++ resolves multiple inheritance ambiguities through virtual inheritance.

The project also introduced professional software engineering workflows: multi-file C++ projects with header guards, GitHub version control, and automated CI/CD via GitHub Actions. Designing the operator overloads for Course and the deep copy semantics for Invoice reinforced the importance of thinking carefully about object ownership and resource management in C++.

Overall, the Smart Campus Management System is a comprehensive demonstration of Object-Oriented Programming in C++, covering every concept from basic encapsulation to advanced virtual inheritance, with clean code, proper memory management, and professional project structure.